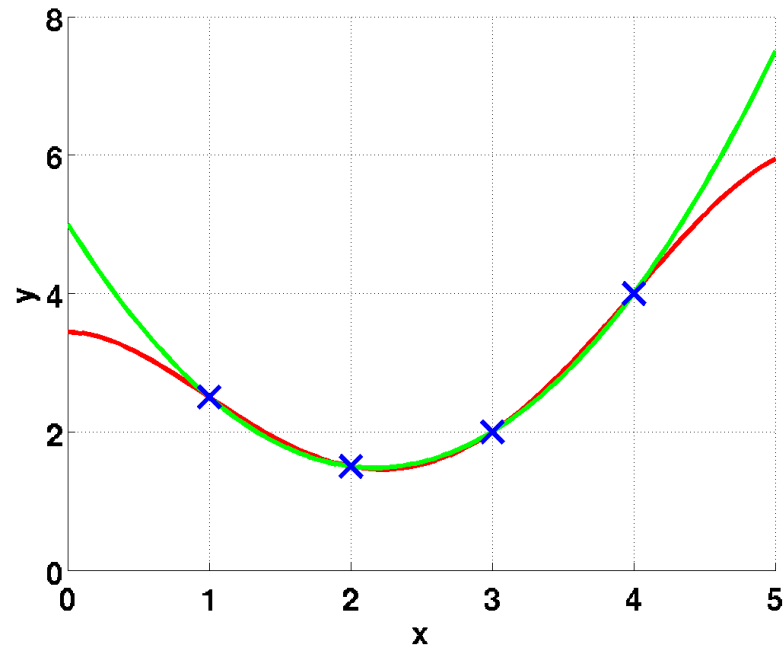


# Computational Engineering

## Interpolation und Approximation

Gegeben sind  $n + 1$  reelle **Wertepaare**  $(x_i, y_i)$ ,  $i = 0, \dots, n$ , mit  $x_i \neq x_j$  für  $i \neq j$  (z.B. Messwerte).



Gesucht ist eine **stetige** Funktion  $f$ , die exakt durch die gegebenen Wertepaare verläuft

$$f(x_i) = y_i.$$

Frage ist: Wie berechnet man “die” Funktion  $f$ ?

**Definition**

Gegeben sind  $n + 1$  reelle Wertepaare  $(x_i, y_i)$ ,  $i = 0, \dots, n$ , mit  $x_i \neq x_j$  für  $i \neq j$ . Eine stetige Funktion  $f$  mit der Eigenschaft

$$f(x_i) = y_i \quad \text{für alle } i = 0, \dots, n$$

heißt **Interpolierende** der Wertepaare  $(x_i, y_i)$ .

Es leuchtet unmittelbar ein, dass es **unendlich viele** Interpolierende zu jeder beliebigen Menge an Wertepaaren gibt. Das Problem eine Interpolierende mittels des Computers zu finden kann nur gelöst werden, wenn zunächst die **Klasse der** in Frage kommenden **Funktionen** geeignet **eingeschränkt** wird!

- Ein **Polynom**  $p$  vom Grad  $n$  ist eine Funktion

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x + a_0.$$

und hat genau  $n + 1$  **Koeffizienten**  $\{a_k, k = 0, \dots, n\}$ .

- Die  $n + 1$  Wertepaare des Interpolationsproblems liefern dann genau genügend viele Bestimmungs-Gleichungen um die  $n + 1$  Koeffizienten eines Polynoms so anzupassen, dass dieses Interpolierende wird.

### Satz

Gegeben sind  $n + 1$  reelle Wertepaare  $(x_i, y_i)$ ,  $i = 0, \dots, n$ , mit  $x_i \neq x_j$  für  $i \neq j$ . Dann gibt es genau ein Polynom  $p$  vom Grad höchstens  $n$ , das **Interpolationspolynom**, mit  $p(x_i) = y_i$  für alle  $i = 0, \dots, n$ .

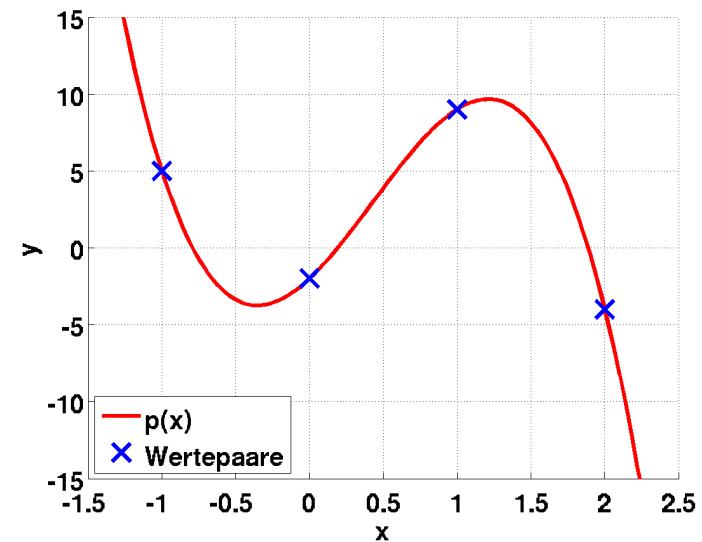
**Beispiel:**

Zu den Wertepaaren

|       |    |    |   |    |
|-------|----|----|---|----|
| $i$   | 0  | 1  | 2 | 3  |
| $x_i$ | -1 | 0  | 1 | 2  |
| $y_i$ | 5  | -2 | 9 | -4 |

Findet sich das **eindeutige**  
**Interpolationspolynom**

$$p(x) = -7x^3 + 9x^2 + 9x - 2.$$

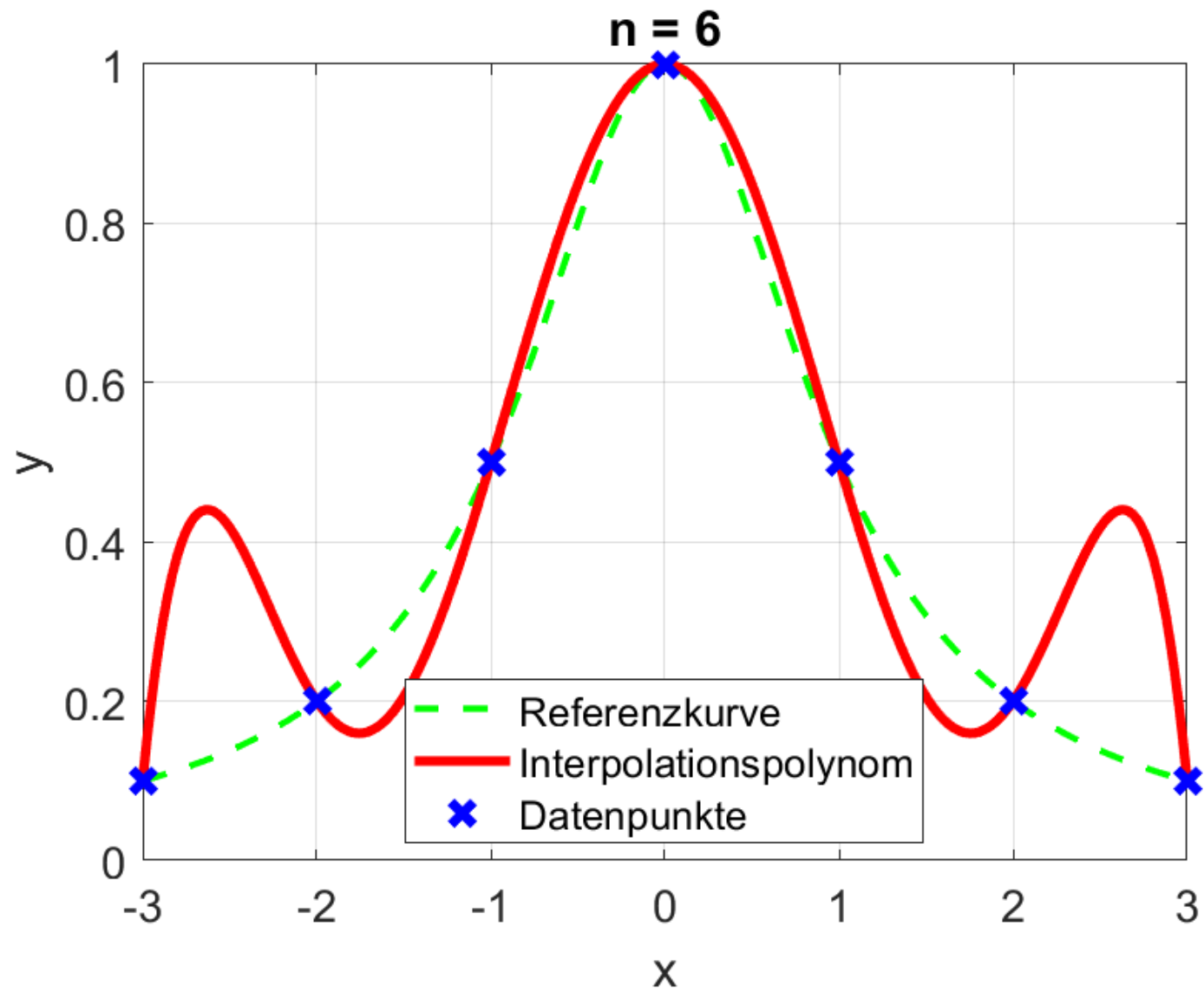


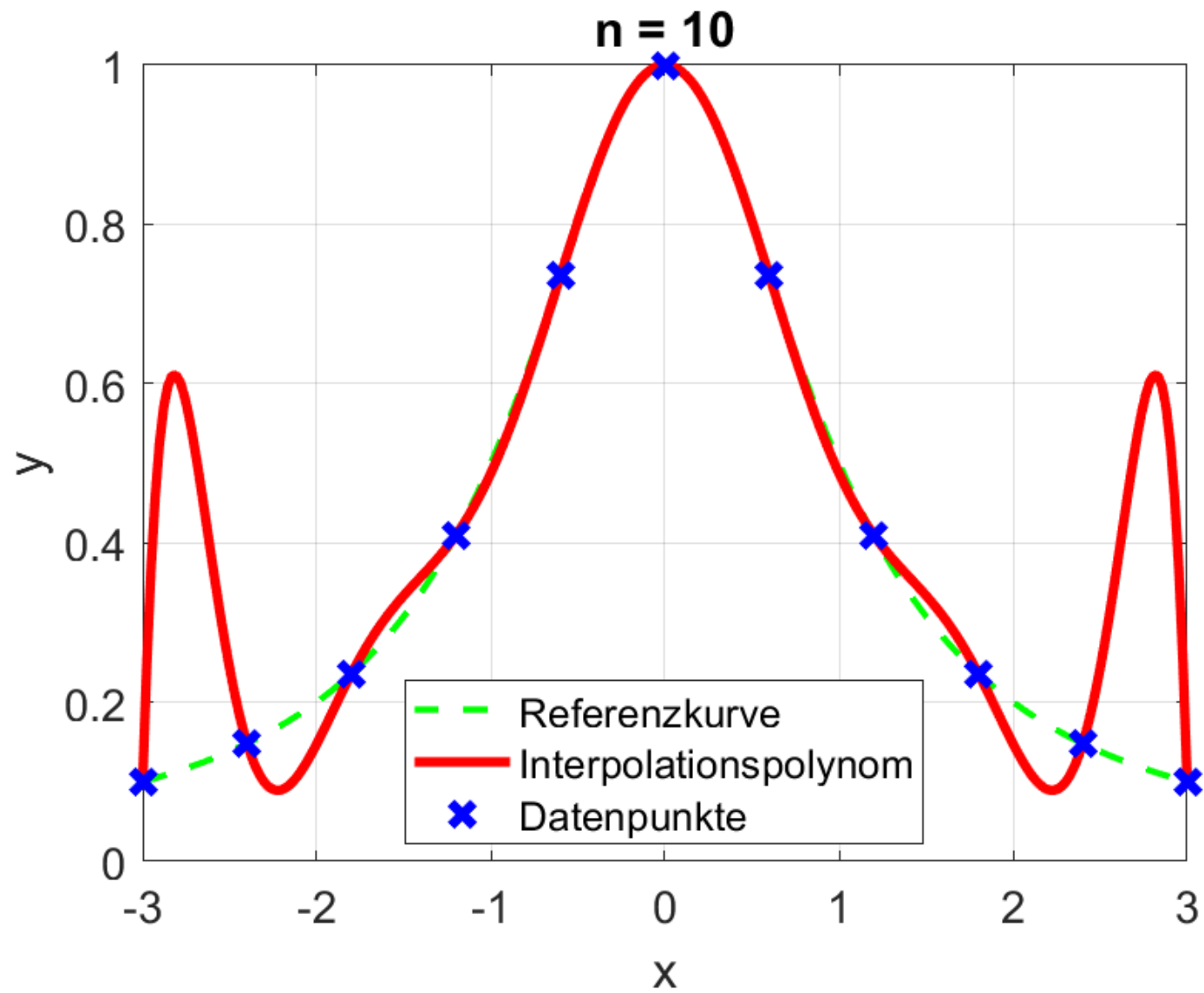
Interpolationspolynome von Datensätzen, insbesondere solche hohen Grades, weisen allerdings ein **grundlegendes Problem auf!** Betrachte dazu den Satz von Wertepaaren

|                |     |     |     |     |     |     |     |
|----------------|-----|-----|-----|-----|-----|-----|-----|
| $i$            | 0   | 1   | 2   | 3   | 4   | 5   | 6   |
| $x_i$          | -3  | -2  | -1  | 0   | 1   | 2   | 3   |
| $y_i = f(x_i)$ | 0.1 | 0.2 | 0.5 | 1.0 | 0.5 | 0.2 | 0.1 |

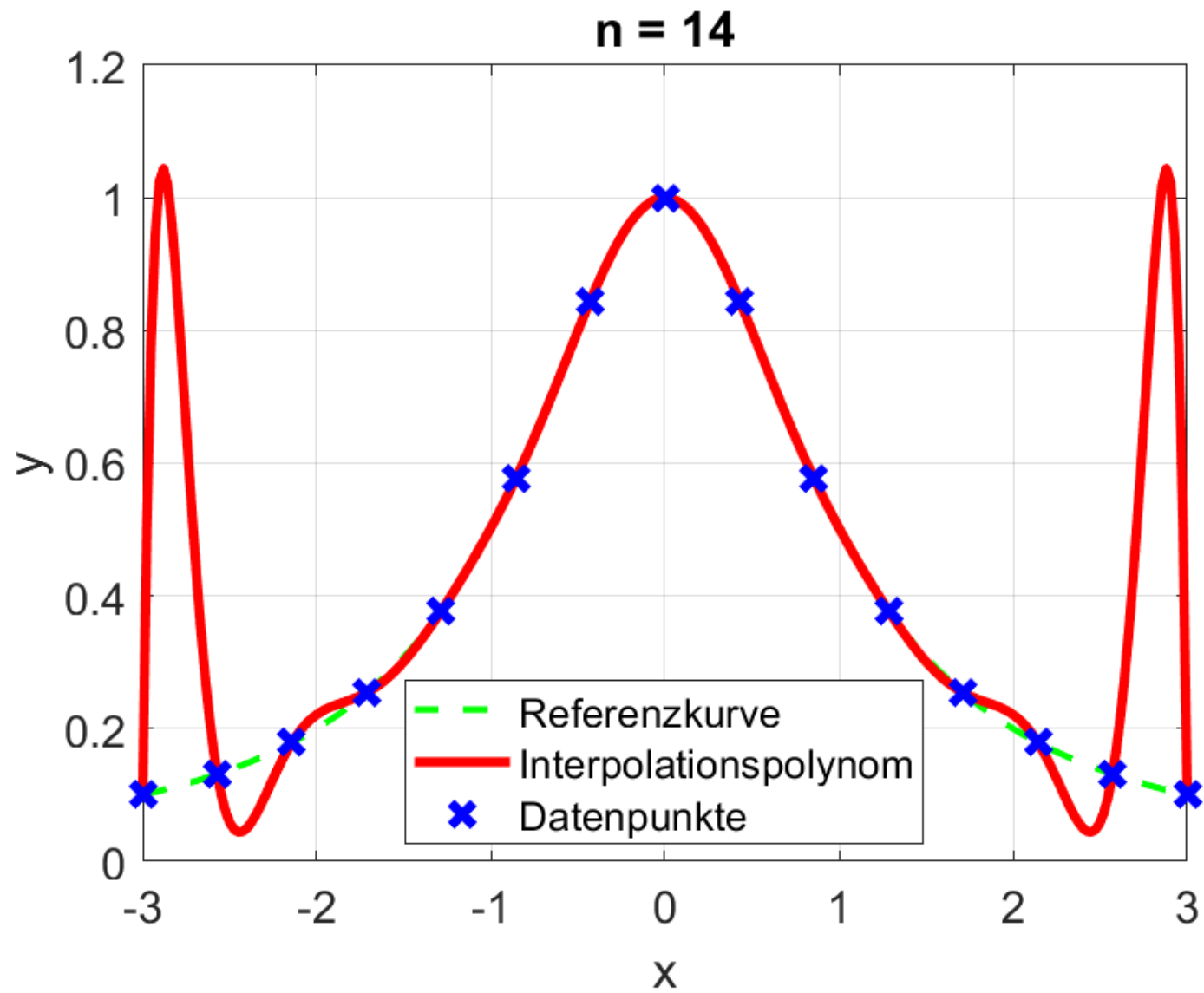
Das Interpolationspolynom errechnet sich hier zu

$$p(x) = -0.01x^6 + 0.15x^4 - 0.64x^2 + 1.$$



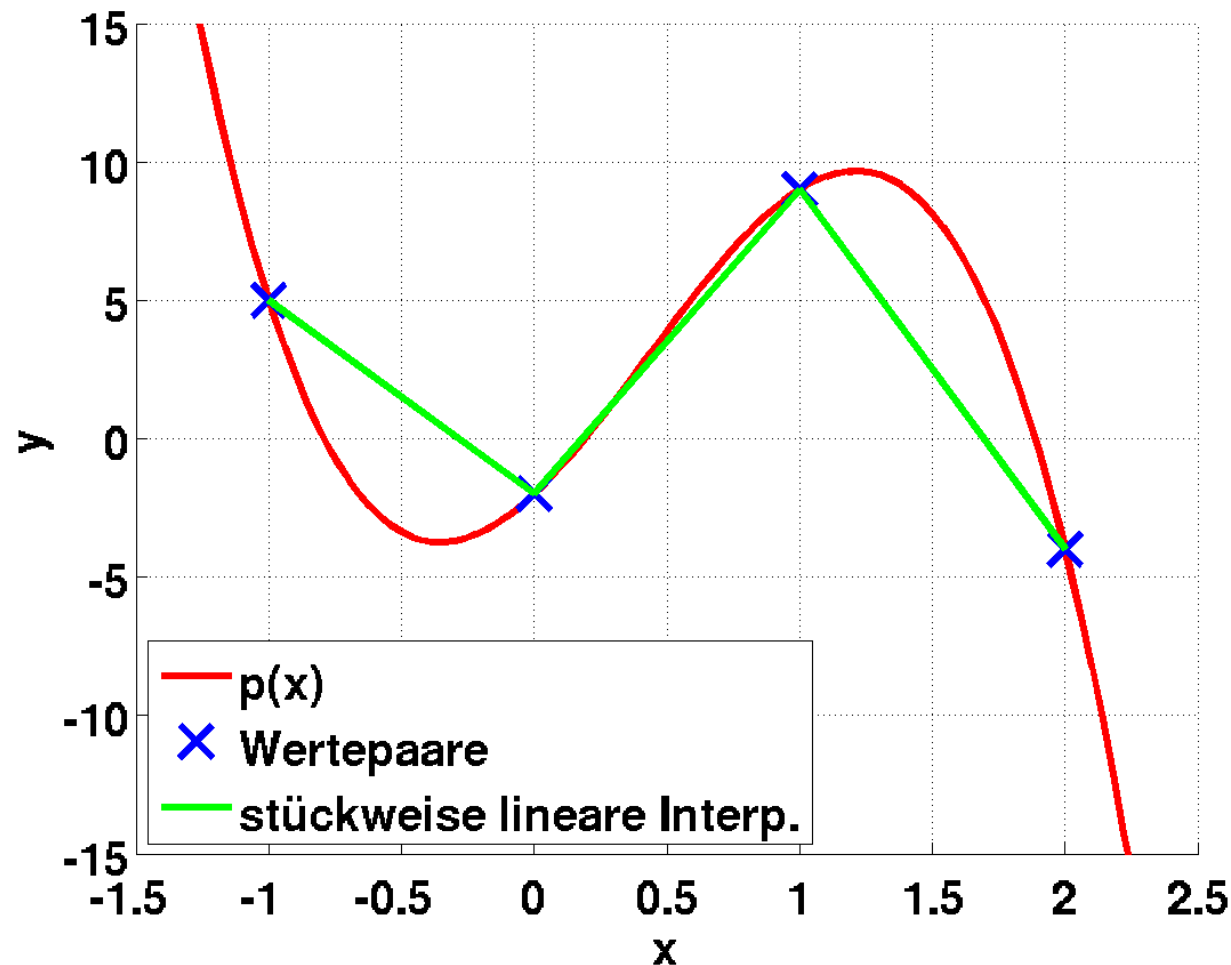






- Je mehr Datenpunkte ein Datensatz hat, desto höheren Grad hat das entsprechende Interpolationspolynom
- Je höher der Grad des Interpolationspolynoms ist, desto stärker werden die “Schwingungen” des Interpolationspolynoms an den Randbereichen des Wertebereichs der Datenpunkte.
- Je größer der Grad eines Interpolationspolynoms ist, desto weniger wird der Verlauf des Polynoms einer den Daten zu Grunde liegenden Funktion (zumindest am Rand des Wertebereichs) sinnvoll folgen.
- Ausweg: **Stückweise** Interpolation mit Polynomen niedrigen Grades!
- Üblich sind Polynome vom Grad 1 ( $\rightarrow$  lineare Interpolation) und Polynome vom Grad 3 ( $\rightarrow$  Spline-Interpolation).

Bei der **stückweisen** Interpolation interpoliert man jeweils nur zwischen **zwei benachbarten** Wertepaaren und setzt die Interpolierenden **stückweise** (stetig!) zusammen. Die einfachste Möglichkeit ist die **stückweise lineare Interpolation**:



## Stückweise lineare Interpolation

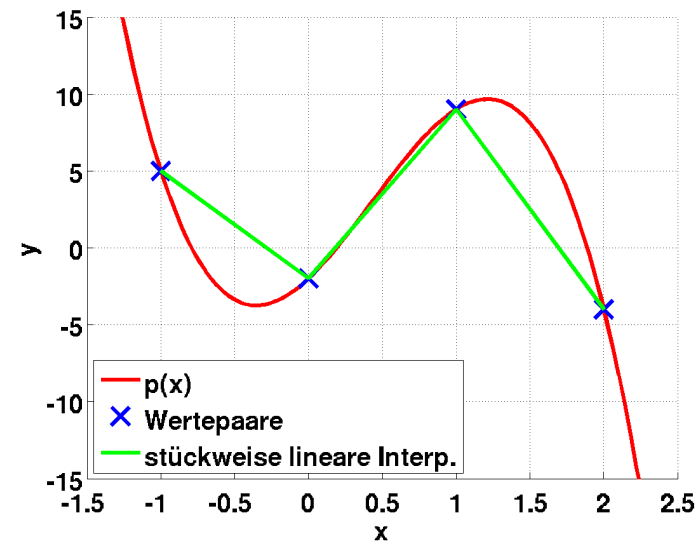
Gegeben sind  $n + 1$  reelle Wertepaare  $(x_i, y_i)$ ,  $i = 0, \dots, n$ , mit  $x_i \neq x_j$  für  $i \neq j$ .

Dann ist die Funktion

$$p(x) = \begin{cases} y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} (x - x_i) & \text{wenn } x \in [x_i, x_{i+1}] \\ 0 & \text{sonst.} \end{cases}$$

die **stückweise lineare Interpolation** der Wertepaare.

Bei der stückweise linearen Interpolation weist die Interpolierende zwangsläufig “Knicke” auf, d.h. die Interpolierende besitzt **keine stetige Ableitung**.

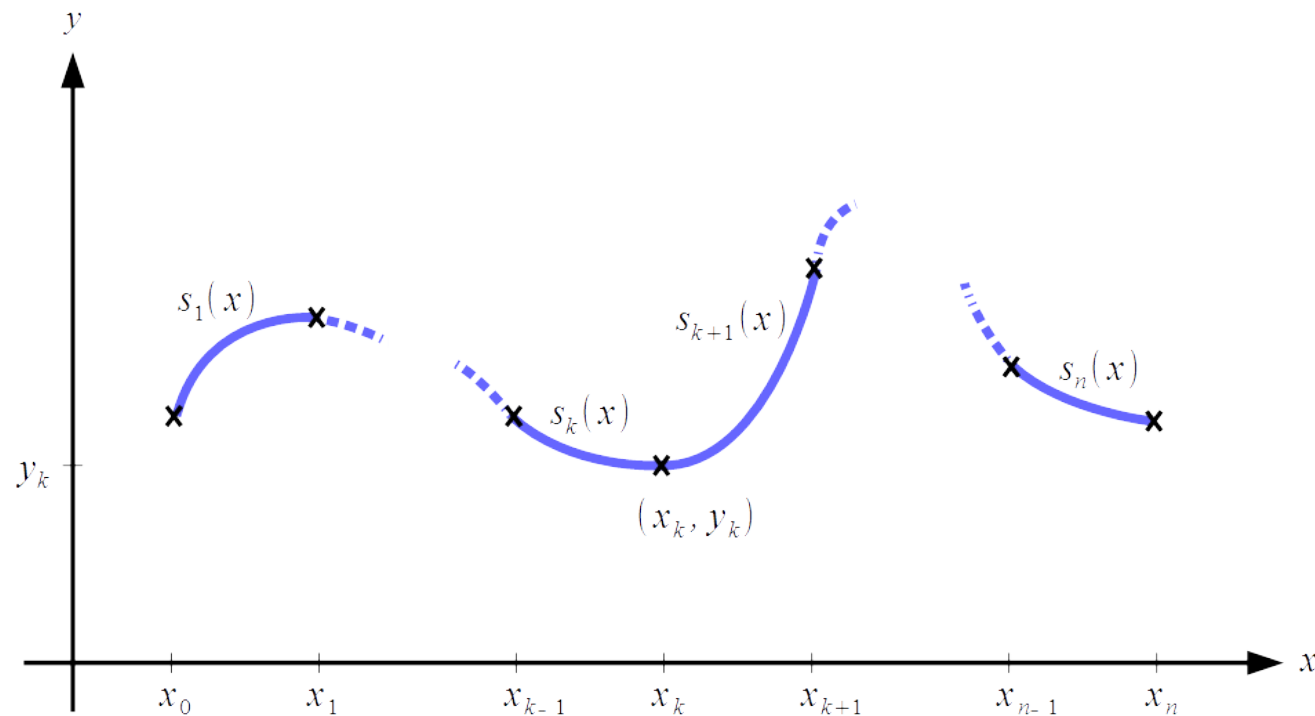


Konstruiert man eine stückweise Interpolierende, welche über dem gesamten Interpolationsintervall sogar noch eine stetige 2. Ableitung besitzt, dann führt dies auf Polynome dritten Grades, die kubischen Splines.

Gegeben sind  $n + 1$  reelle Wertepaare  $(x_i, y_i)$ ,  $i = 0, \dots, n$ , mit  $x_i \neq x_j$  für  $i \neq j$ . Diese interpoliert man stückweise mit  $n$  **kubischen Polynomen**

$$s_k(x) = a_k x^3 + b_k x^2 + c_k x + d_k, \quad x \in [x_{k-1}, x_k]$$

für  $k = 1, \dots, n$ :



## Die $4n$ unbestimmten Koeffizienten

$$\{a_k, b_k, c_k, d_k\}, \quad k = 1, \dots, n$$

der  $n$  kubischen Polynome werden festgelegt durch die Forderungen:

- Die Interpolierende geht exakt durch jedes Wertepaar und ist **stetig** in den Klebestellen:

$$s_k(x_{k-1}) = y_{k-1} \quad \text{und} \quad s_k(x_k) = y_k \quad \text{für } k = 1, \dots, n.$$

Das sind  $2n$  Bestimmungsgleichungen.

- Die Interpolierende besitzt eine stetige erste und zweite Ableitung an den Klebestellen

$$s'_k(x_k) = s'_{k+1}(x_k) \quad \text{und} \quad s''_k(x_k) = s''_{k+1}(x_k) \quad \text{für } k = 1, \dots, n-1.$$

Das sind  $2n - 2$  Bestimmungsgleichungen.

- Es fehlen noch 2 weitere Gleichungen um die  $4n$  Koeffizienten eindeutig festzulegen. Dafür gibt es mehrere Möglichkeiten - üblich in MATLAB ist die "not-a-knot"-Variante:  $s'''(x)$  ist stetig an  $x_1$  und  $x_{n-1}$ .

**Definition: kubischer Spline**

Gegeben sind  $n + 1$  reelle Wertepaare  $(x_i, y_i)$ ,  $i = 0, \dots, n$ , mit  $x_i \neq x_j$  für  $i \neq j$ . Die Funktion

$$s(x) = \begin{cases} s_1(x), & \text{für } x \in [x_0, x_1] \\ s_2(x), & \text{für } x \in [x_1, x_2] \\ \vdots & \\ s_n(x), & \text{für } x \in [x_{n-1}, x_n] \end{cases}$$

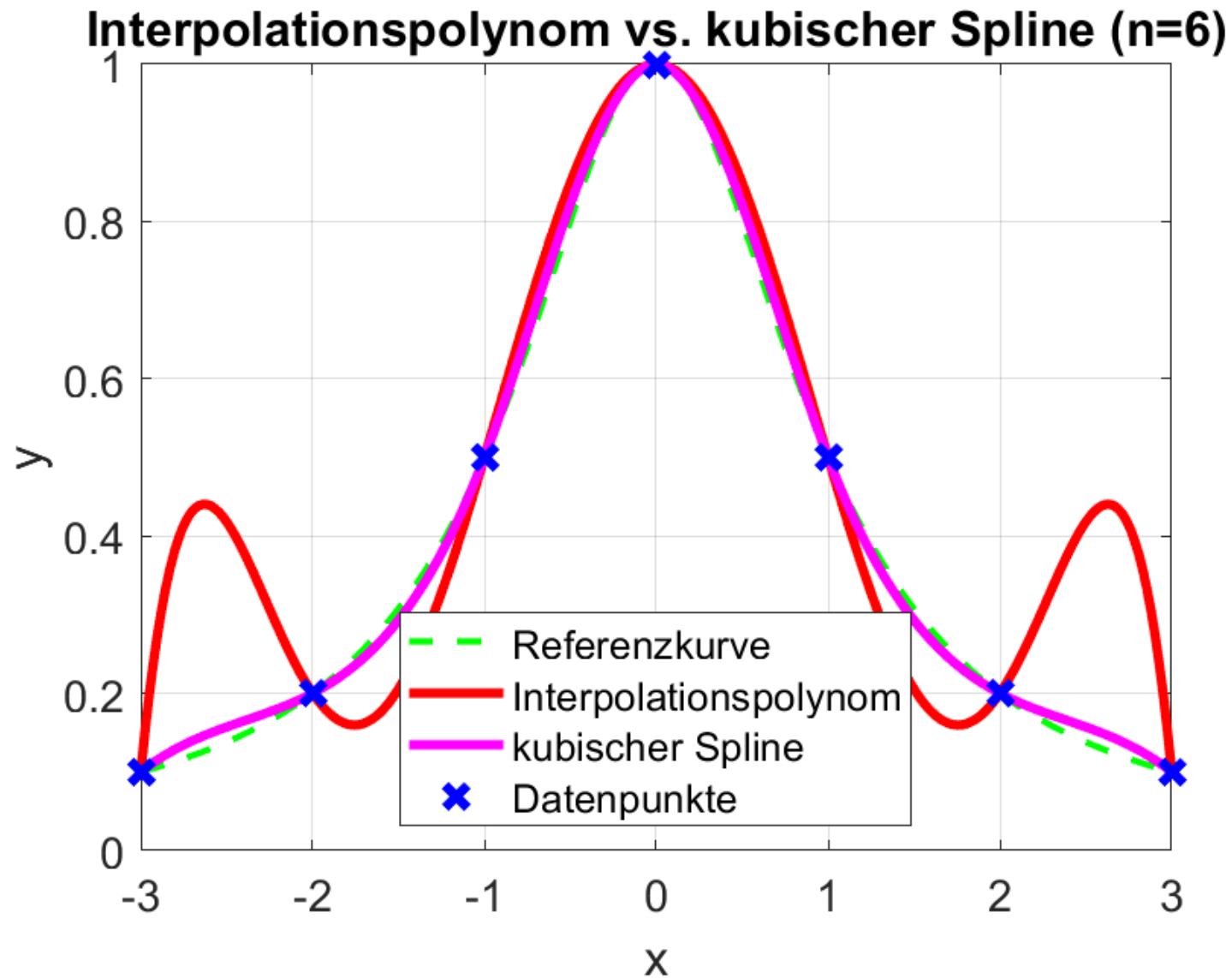
mit

$$s_k(x) = a_k x^3 + b_k x^2 + c_k x + d_k, \quad k = 1, \dots, n$$

die zusammen mit ihrer ersten und zweiten Ableitung auf dem gesamten Intervall  $[x_0, x_n]$  stetig ist und deren dritte Ableitung zusätzlich an den Punkten  $x_1$  und  $x_{n-1}$  stetig ist, nennt man den **kubischen interpolierenden Spline** der Datenpaare.



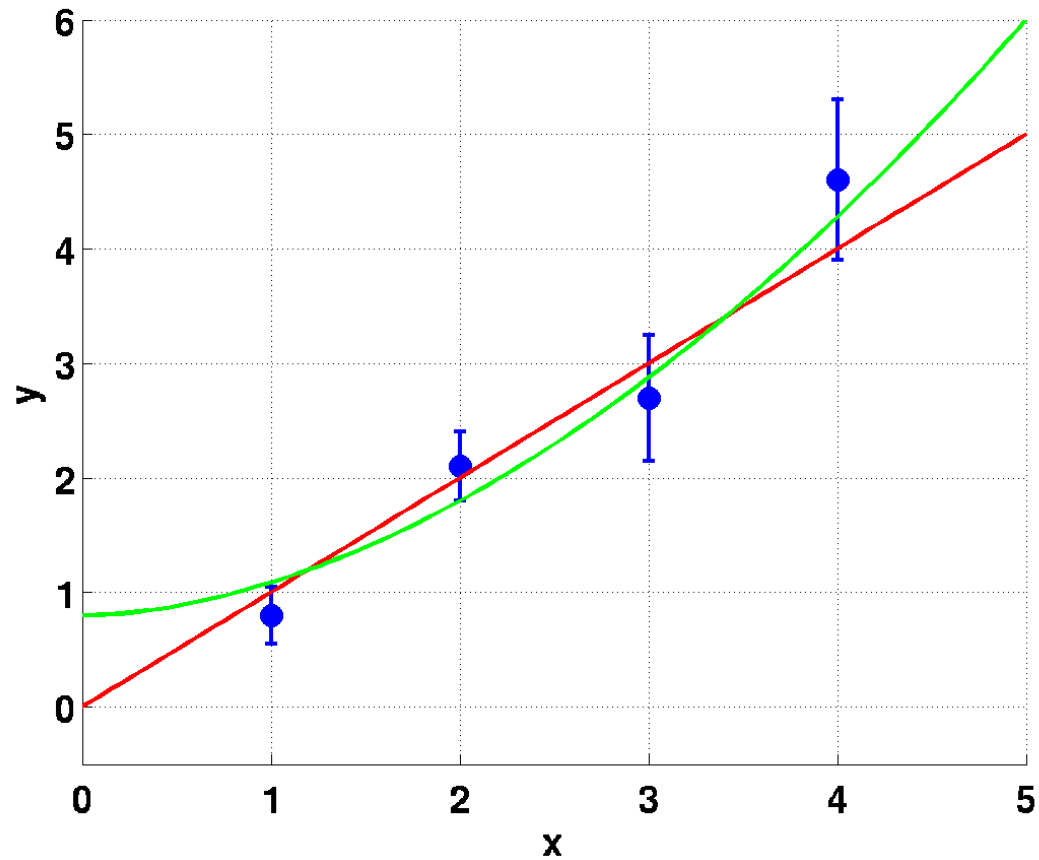
Beispiel:



Eine etwas andere Problemstellung: Gegeben sind  $n$  Wertepaare  $(x_i, y_i)$ ,  $i = 1, \dots, n$  mit  $x_i \neq x_j$  für  $i \neq j$ .

- Bei der Interpolation wurde eine Funktion gesucht, die dem “Verlauf” der Wertepaare bestmöglich folgt und dabei die Wertepaare exakt reproduziert.
- In der Anwendung sind die Wertepaare häufig intrinsisch mit einem Fehler behaftet (Messungenauigkeit etc.), so dass es nicht sinnvoll ist, dass die annähernde Funktion die Datenpunkte genau durchläuft.
- Man sucht deshalb besser eine stetige Funktion  $f$ , die in einem gewissen Sinn “bestmöglich” die Wertepaare annähert und dabei möglichst nahe an diesen liegt, d.h. für die  $f(x_i) \approx y_i$  für alle  $i$  gilt. Das nennt man **Approximation**.
- Auch hier muss man die Menge der Funktionen unter denen man die Funktion  $f$  sucht zunächst geeignet einschränken.

Welche Funktion approximiert die Wertepaare nun am “besten”?



**Definition**

Gegeben sei eine Menge  $\mathcal{F}$  von stetigen Funktionen (die Ansatzfunktionen) definiert auf einem Intervall  $[a, b]$ :  $\mathcal{F} \subset C([a, b])$ , sowie  $n$  Wertepaare  $(x_i, y_i)$ ,  $i = 1, \dots, n$ .

Eine Funktion  $f \in \mathcal{F}$  heißt Ausgleichsfunktion **im quadratischen Mittel** aus  $\mathcal{F}$  zu den gegebenen Wertepaaren, falls das Fehlerfunktional

$$E[f] = \sum_{i=1}^n (y_i - f(x_i))^2$$

für  $f$  minimal wird, d.h.

$$E[f] = \min_{g \in \mathcal{F}} (E[g])$$

Dies ist die Formulierung des **allgemeinen Ausgleichsproblems**.

Das allgemeine nichtlineare Ausgleichsproblem ist offenbar eine **Minimierungsaufgabe** und wird in der Praxis mit numerischen **Optimierungsalgorithmen** gelöst.

- Die Ansatzfunktionen werden idR so angesetzt, dass diese eine Reihe **freier Parameter**  $\lambda_1, \lambda_2, \lambda_3, \dots$  (im Sinne einer Funktionenschar) enthalten

$$f = f(x, \lambda_1, \lambda_2, \lambda_3, \dots) = f(x, \vec{\lambda})$$

unter denen dann derjenige Satz von Parametern gesucht wird, der das Minimierungsproblem löst.

- Typischerweise wird das Minimum des Fehlervektors

$$\vec{\delta}(\vec{\lambda}) = (f(x_1, \vec{\lambda}) - y_1, f(x_2, \vec{\lambda}) - y_2, \dots, f(x_n, \vec{\lambda}) - y_n)^T$$

bzgl. der 2-Norm gesucht. Andere Normen sind möglich, können aber zu abweichenden Ergebnissen führen.

- Es ist möglich einzelne Messwerte mit unterschiedlichem Gewicht zu berücksichtigen, d.h. das Funktional

$$\tilde{E}[f] = \sum_{i=0}^n w_i (f(x_i, \vec{\lambda}) - y_i)^2$$

zu minimieren.

**Allgemeines (i. A. nichtlineares) Ausgleichsproblem**

Gegeben sei eine Menge von Ansatzfunktionen  $f$ , die von der Variable  $x$  und von  $m$  Parametern  $\lambda_1, \lambda_2, \dots, \lambda_m$  abhängen.

$$\mathcal{F} = \{f(x, \lambda_1, \lambda_2, \dots, \lambda_m) \mid \lambda_j \in \mathbb{R} \text{ für alle } j = 1, \dots, m\},$$

sowie  $n$  Wertepaare  $(x_i, y_i)$ ,  $i = 1, \dots, n$ . Das quadratische Fehlerfunktional lautet

$$E(\lambda_1, \dots, \lambda_m) = \sum_{i=1}^n (y_i - f(x_i, \lambda_1, \dots, \lambda_m))^2$$

und gesucht ist die Lösung des Optimierungsproblems

$$\min_{\vec{\lambda} \in \mathbb{R}^m} E(\lambda_1, \dots, \lambda_m)$$

wobei eventuelle Einschränkungen erlaubter Bereiche für die Werte der  $\lambda_j$  zu berücksichtigen sind.

Die Lösung des gerade besprochenen Ausgleichsproblems ist technisch die Lösung eines Minimierungsproblems, d.h. wir suchen für eine gegebene  $n$ -dimensionale reellwertige Funktion  $f(\vec{x}) = f(x_1, x_2, \dots, x_n)$  einen Punkt  $\vec{x}^* \in \mathbb{D} \subseteq \mathbb{R}^n$  an dem die Funktion  $f$  ein Minimum aufweist. Hierfür gibt es eine ganze Reihe numerischer Verfahren, die sich je nach Art der Funktion und der Menge  $\mathbb{D}$  dafür eignen.

In der Übung wollen wir uns mit den beiden in Matlab vorimplementierten Verfahren

- `fminunc`
- `fminsearch`

beschäftigen und diese an einigen Beispielen ausprobieren.